



Grado en Intelixencia Artificial
Sistemas Multiagente

Práctica 2: Comunicación y comportamiento social

Miguel Baños Baladrón
Fiz Garrido Escudero
Grupo Martes

3 de Mayo de 2026

Contents

1	Mundo Virtual	3
2	Arquitectura individual	5
3	Comunicación entre agentes	12
4	Formato del contenido de los lenguajes	13
5	Estrategia Multiagente	15

Mundo Virtual

El mundo virtual mantiene la estructura general de la Práctica 1: un exterior rodeando un castillo central, con guardias estáticos en la puerta y patrulleros en el interior, un cofre que el ladrón debe robar y una zona de salida por la que escapar.

Las novedades del escenario son:

- **Rejas dinámicas:** obstáculos cíclicos que emergen del suelo durante un tiempo y luego se retraen. Mientras están elevadas, tallan el NavMesh con un `NavMeshObstacle`, bloqueando temporalmente rutas que antes eran transitables. Esto obliga a los patrulleros a replantear sus rutas en tiempo de ejecución.

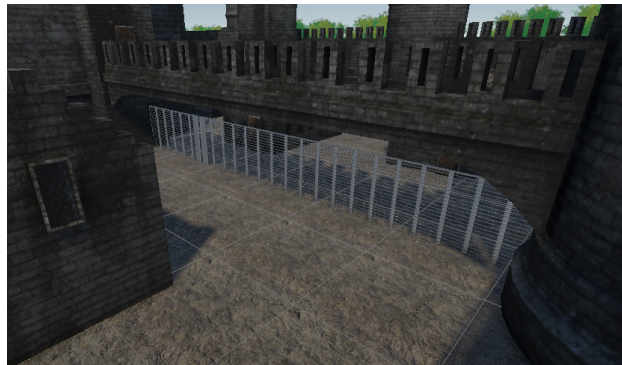


Figure 1: Reja dinámica emergiendo del suelo.

- **Cámaras de vigilancia:** nuevos agentes estáticos colocados en puntos estratégicos del castillo que sólo perciben (no se mueven ni atacan) y coordinan a los patrulleros para investigar avistamientos.



Figure 2: Cámara de vigilancia colocada en el castillo.

- **Puntos de entrada** para los guardias estáticos: GameObjects vacíos colocados dentro del NavMesh cerca de la puerta, asignados en el Inspector. Sirven como referencia geográfica accesible cuando el guardián exterior debe pedir ayuda al interior (la posición real del ladrón puede estar fuera del NavMesh de los patrulleros).

El resto del escenario (terreno, vegetación, distribución de casas y torreón, restricciones de movimiento por tipo de agente) se mantiene exactamente igual que en la Práctica 1.

Agentes del mundo

El sistema cuenta con tres tipos de agente comunicante, además del ladrón controlado por el jugador.

Patrullero Es el agente más completo. Dispone de las cinco partes de la arquitectura: sensores (visión, escucha, movimiento, hitbox), reactiva, deliberativa, social y actuador. Su interacción con el entorno es doble: percibe al ladrón mediante visión y escucha, y detecta cambios en el estado del cofre y las rejas dinámicas. Puede ser tanto Iniciador como Participante en cualquier protocolo: comprobaciones periódicas del cofre (por timer), investigación de avistamientos comunicados por una cámara, respuesta a alertas

de un guardián estático, bloqueo de la zona de salida tras detectar el robo y búsqueda coordinada tras perder al ladrón.

Guardián Vigila la entrada del castillo. No puede moverse al interior y por ello siempre responde **REFUSE** a cualquier **REQUEST** que reciba. Sí puede ser **Iniciador** en una situación:

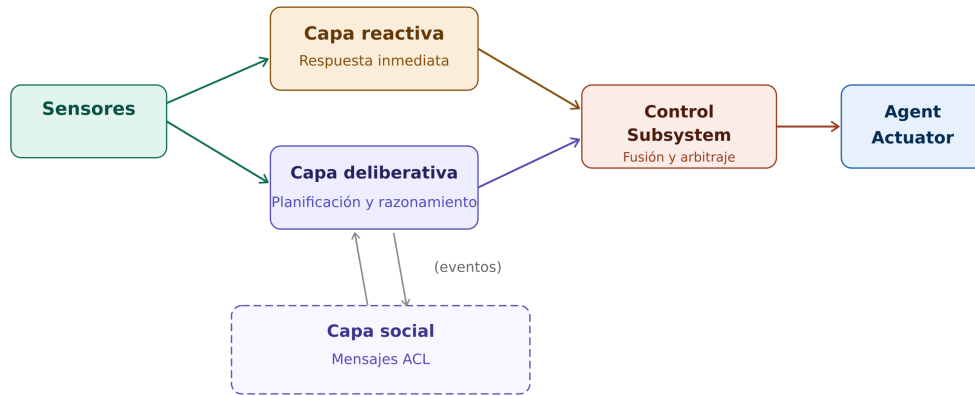
- **Por avistamiento:** si detecta al ladrón con su **VisionSensor**, lanza la tarea **ladronAvistado** con la posición de su **puntoEntrada** (un **GameObject** vacío dentro del **NavMesh**) como referencia, para que un patrullero acuda a esa zona.

Cámara Agente exclusivamente perceptivo y comunicativo. No tiene capa reactiva, ni **NavMeshAgent**, ni **HearingSensor**, ni **NoiseEmitter**. Su único sensor es el **CameraSensor**, una versión simplificada del **VisionSensor** que sólo detecta al ladrón (sin lógica de cofre, ni reescaneo de entorno). Cuando lo detecta, su deliberativa activa el flag **protocoloActivo** y lanza un **Contract Net** con tarea **investigarAvistamiento**. Como nunca puede ser **Participante**, su **CameraSocialLayer** sólo implementa el vocabulario del **Iniciador**.

Arquitectura individual

Se conserva la arquitectura híbrida en capas inspirada en **TouringMachines** de la Práctica 1, pero ampliada con una nueva capa **Social** que se sitúa por encima de las capas reactiva y deliberativa. La regla de inhibición sigue intacta: la capa reactiva siempre tiene prioridad si solicita el control.

El flujo de datos completo queda:



Las capas reactiva y deliberativa nunca manipulan mensajes ACL. La deliberativa habla con la capa social mediante eventos y llamadas a métodos del lenguaje del juego (Ej: “enviar REQUEST con tarea X”, “he recibido una propuesta del agente Y con coste Z”). La capa social es la única que conoce el formato y el vocabulario de la comunicación.

Esta separación permite que cámaras y esqueletos compartan exactamente la misma infraestructura de mensajería sin compartir reactiva ni capa de movimiento.

Se han corregido también varios errores y limitaciones de la Práctica 1:

- **Cesión del control:** la capa reactiva del patrullero ya no monopoliza el control durante la patrulla. `PatrolBehaviour` devuelve `null` mientras el `NavMeshAgent` se desplaza por inercia, cediendo el control a la deliberativa para que gestione el protocolo Contract Net. En consecuencia, el árbitro (`ControlSubsystem`) también se ha corregido para tolerar propuestas `null` de cualquiera de las dos capas.
- **Modelo de percepción basado en eventos:** en la Práctica 1 las capas consultaban activamente a los sensores cada frame (modelo *pull*). Ahora son los sensores quienes notifican a los behaviours mediante eventos `C#` cuando detectan un cambio relevante: el ladrón es visto o perdido, se escucha un ruido, el agente llega a su destino, el cofre desaparece, etc. Los behaviours se suscriben en `Initialize()` y mantienen su propio estado interno. Esto elimina el acoplamiento directo entre

capas y sensores, y garantiza que ninguna información se pierde entre frames.

Capa Reactiva

Igual que en la Práctica 1, la capa reactiva consulta a una cadena priorizada de *behaviours* y devuelve la primera propuesta no nula. La novedad es que ahora puede devolver `null` legítimamente (entre puntos de patrulla, mientras el NavMeshAgent se desplaza por inercia), cediendo así el control a la deliberativa.

Existen dos especializaciones:

- **PatrolReactiveLayer**: cadena *Attack ¿ Chase ¿ Investigate ¿ FollowGuard ¿ Patrol*.
- **GuardianReactiveLayer**: cadena *Attack ¿ Chase ¿ Investigate ¿ FollowGuard ¿ Watch*.

La cámara **no tiene capa reactiva**: la capa reactiva existe para elegir entre behaviours de movimiento y ataque según los estímulos del momento (perseguir, atacar, investigar...). La cámara no se mueve ni ataca, así que no tiene ningún behaviour que ejecutar. Si tuviera capa reactiva, simplemente devolvería `null` cada frame sin hacer nada útil. Cuando el sensor detecta al ladrón, la única respuesta posible es coordinar a otros agentes mediante el protocolo Contract Net, que es responsabilidad de la capa deliberativa. Por eso el evento del sensor va directamente a la deliberativa, eliminando una capa que no aportaría nada.

Capa Deliberativa

La capa deliberativa implementa la **máquina de estados (FSM)** del protocolo Contract Net. Cada tipo de agente tiene su propia FSM:

- **PatrolDeliberativeLayer** actúa como **Iniciador y Participante**. Sus estados son: `IDLE` (sin conversación activa, esperando el timer o un evento), `INIT_WAIT_RESPONSES` (REQUEST enviado, esperando AGREE/REFUSE de los demás agentes), `INIT_WAIT_PROPOSALS` (CFP enviado a los voluntarios, esperando sus PROPOSEs), `INIT_WAIT_RESULT` (ganador aceptado, esperando su INFORM.RESULT), `PART_VOLUNTEERED` (ha respondido AGREE, esperando el CFP del iniciador), `PART_PROPOSED` (ha enviado su propuesta, esperando ACCEPT o REJECT_PROPOSAL)

y `PART_EXECUTING` (ha ganado el contrato y está ejecutando el behaviour comprometido). Maneja las tareas `comprobarCofre`, `investigarAvistamiento`, `ladronAvistado`, `bloquearSalida` y `busquedaCoordinada`.

- `GuardianDeliberativeLayer` actúa sólo como **Iniciador**, con los mismos estados `IDLE`, `INIT_WAIT_RESPONSES`, `INIT_WAIT_PROPOSALS` e `INIT_WAIT_RESULT`. Como participante siempre responde `REFUSE` (no puede moverse al interior del castillo). Lanza protocolos de tarea `ladronAvistado`.
- `CameraDeliberativeLayer` actúa sólo como **Iniciador**. No hereda de `DeliberativeLayer` porque la cámara nunca propone movimiento al `ControlSubsystem`. Sus estados son equivalentes a los del iniciador del patrullero pero con nombres simplificados: `IDLE`, `WAIT_RESPONSES`, `WAIT_PROPOSALS` y `WAIT_RESULT`.

Cuando la deliberativa está en estado `PART_EXECUTING` (el agente ha ganado el contrato y está cumpliendo su compromiso) su método `GenerarPropuesta()` devuelve un `ActionProposal` delegando en el behaviour comprometido (`CheckChestBehaviour` o `GoToPositionBehaviour`). En cualquier otro estado devuelve `null`.

Capa Social

La capa social es completamente nueva en esta práctica. Se compone de tres elementos:

AgenteFIPAACL Clase abstracta de la que heredan todas las capas sociales. Aporta:

- Registro estático `Todos` con todas las instancias activas en escena, actualizado en `OnEnable/OnDisable`.
- `Inbox` (cola de mensajes) drenado en `Update` con un límite `mensajesPorFrame` configurable (por defecto 2) para evitar congelar el juego ante avalanchas de mensajes.
- Envío únicamente **unicast**: cualquier multicast se hace por bucle explícito desde la capa social.
- Factoría `CrearMensaje` que rellena `sender` y genera un GUID para `conversationId`.
- Hook abstracto `ProcesarMensaje(ACLMessage)` que cada subclase implementa con su vocabulario.
- Historial de mensajes requerido por el enunciado de la práctica.

SkeletonSocialLayer Hereda de **AgenteFIPAACL** y define el lenguaje completo del protocolo Contract Net, las estructuras de datos del mensaje, la API de métodos que llama la deliberativa (**EnviarRequest**, **EnviarCFP**, **EnviarPropuesta**, **InformarResultado**, **BroadcastInform**, etc.) y los eventos que la deliberativa escucha (**OnRequestReceived**, **OnCFPReceived**, **OnProposalReceived**, etc.). También gestiona el **timer aleatorio coordinado**: cada agente sorteia un valor en `[timerMin, timerMax]`, se pausa al entrar en una conversación (**TIMER_MAX**) y se reinicia al cierre, sincronizando emergentemente quién intenta iniciar a continuación (usado en la tarea de revisar el cofre). Los mensajes con performative desconocido o **content** de tipo incorrecto generan automáticamente un **NOT_UNDERSTOOD** al emisor.

CameraSocialLayer Hereda de **AgenteFIPAACL** e implementa únicamente el vocabulario necesario para el rol de Iniciador: **REQUEST**, **CFP**, **ACCEPT_PROPOSAL**, **REJECT_PROPOSAL** y **REFUSE**. Utiliza las mismas estructuras de datos que **SkeletonSocialLayer** para que los guardias puedan interpretar correctamente los mensajes recibidos de la cámara — si la cámara definiera estructuras propias con campos idénticos, los guardias no las reconocerían al ser tipos distintos en tiempo de ejecución.

Sensores

VisionSensor Heredado de la Práctica 1 con varias mejoras. Detecta al ladrón mediante distancia, ángulo y *raycast* de línea de visión. Las novedades son:

- **Escaneo del entorno** en 16 direcciones, usado por **PatrolBehaviour** para planificar rutas.
- **Detección de cambios en rejas dinámicas**: cada frame consulta la lista estática `DynamicBars.todas` y, si una reja visible cambia de estado, relanza `EscanearEntorno()` para que el patrullero replantee su ruta.
- **Comprobación del cofre**: cuando `GameManager.hasLoot == true` y el cofre está dentro del cono de visión, dispara `OnCofresDesaparecido` una sola vez. Esto se usa ahora en el patrullero para lanzar el protocolo `bloquearSalida`.
- **Método público** `ComprobarEstadoCofre()` que devuelve `true` si el cofre sigue intacto, consultado por `CheckChestBehaviour` al llegar al cofre para determinar el resultado de la comprobación.

- **Robustez:** null-checks en `eyePoint`, `cofre` y `MovementSensor` para que pueda usarse en agentes que no tengan todos esos componentes.
- Re-escaneo automático al llegar a destino mediante suscripción `MovementSensor.OnDestinoAlcanzado`.

HearingSensor Heredado de la Práctica 1. Detecta emisores de ruido (`NoiseEmitter`) en un radio configurable, distingue al ladrón (tag `Ladron`) de otros guardias (tag `Esqueleto`) y aplica una imprecisión configurable a la posición percibida. Dispara `OnLadronEscuchado`, `OnEsqueletoEscuchado` y sus correspondientes eventos de pérdida.

MovementSensor Sin cambios. Monitoriza el `NavMeshAgent` y dispara `OnDestinoAlcanzado` cuando el agente llega a su destino con una tolerancia de 0.3 unidades. Es el evento clave que cierra los behaviours `CheckChestBehaviour` y `GoToPositionBehaviour`.

SwordHitBox Sin cambios. Sigue actuando como sensor de impacto y reiniciando la escena al colisionar con el ladrón.

CameraSensor Nuevo en esta práctica. Implementación reducida al mínimo: detecta al ladrón con los tres mismos criterios (distancia, ángulo, raycast) y dispara `OnLadronAvistado` y `OnLadronPerdido` en los flancos de cambio de estado. Expone `PosicionLadron` como propiedad pública. El gizmo dibuja el cono 3D real usando `transform.right` y `transform.up` para reflejar la orientación de la cámara. No incluye lógica de cofre, rejas ni escaneo de entorno, ya que la cámara no las necesita.

Behaviours

Los behaviours siguen siendo clases C# que implementan la interfaz `IBehaviour` con un método `Evaluate() : ActionProposal?`. Se distinguen dos categorías según quién los activa.

Behaviours reactivos Son los que viven en la cadena priorizada de la capa reactiva y responden directamente a estímulos sensoriales.

- **AttackBehaviour:** ladrón visible y a distancia de ataque.
- **ChaseBehaviour:** ladrón visible pero fuera de alcance.

- **InvestigateBehaviour**: ladrón escuchado pero no visto, va a la posición del ruido a velocidad de caminar.
- **FollowGuardBehaviour**: otro guardia ha sido escuchado corriendo y no hay información directa del ladrón.
- **PatrolBehaviour**: *fallback* del patrullero. Modificado en esta práctica para devolver `null` mientras el `NavMeshAgent` ya tiene un destino activo, cediendo así el control a la deliberativa mientras patrulla. Sólo devuelve un `ActionProposal` al recalcular un nuevo destino (al llegar al anterior o al detectar un cambio de entorno). Además expone el campo `velocidadAlerta` (3 m/s por defecto) y el método `ActivarModoAlerta()` para que la deliberativa pueda subir la velocidad de patrulla cuando se confirma el robo del cofre.
- **WatchBehaviour**: *fallback* del guardián estático, oscila la mirada $\pm \text{watchAngle}^{\circ}$.

Behaviours activados por la deliberativa Una segunda familia de behaviours no participa en la cadena reactiva. Los activa explícitamente la capa deliberativa al ganar un contrato y representan el **compromiso** adquirido.

- **CheckChestBehaviour**: activado al ganar el contrato `comprobarCofre`. Mueve al guardia hasta una distancia de parada del cofre. Al llegar consulta `VisionSensor.ComprobarEstadoCofre()` y dispara el evento `OnComprobacionCompletada(bool cofreIntacto)`.
- **GoToPositionBehaviour**: behaviour genérico que desplaza al agente a un `Vector3` arbitrario en lugar de a un `Transform` fijo. Dispara `OnLlegadaCompletada`. Es compartido por cuatro tareas distintas: `investigarAvistamiento` (cámara), `ladronAvistado` (guardián exterior), `bloquearSalida` (patrullero que ha visto el cofre vacío) y `busquedaCoordinada` para llevar a cada patrullero a su sector.

Árbitro y actuador

ControlSubsystem Sigue siendo el árbitro entre las dos capas de decisión. La única modificación respecto a la Práctica 1 es que ahora tolera propuestas `null` en cualquiera de las dos capas, requisito imprescindible para que el patrullero pueda ceder el control a la deliberativa mientras patrulla. La regla de inhibición para la reactiva se mantiene.

AgentActuator Sin cambios respecto a la Práctica 1. Sigue siendo el componente que ejecuta las decisiones (**MoverA**, **Rotar**, **IniciarAtaque**) sobre el **NavMeshAgent** y el **Animator**, sin tomar decisiones propias. Mantiene el cooldown de ataque y la suspensión del control durante la animación.

Movimiento del ladrón

El ladrón sigue siendo el único agente del juego que no usa **NavMeshAgent**: se controla mediante un **CharacterController** con entrada WASD desde el sistema **InputSystem.Actions**. Esta separación es deliberada — el ladrón es controlado por el jugador, no es un agente autónomo del SMA — y se mantiene exactamente igual que en la Práctica 1.

El ladrón comparte con los guardias el componente **NoiseEmitter**, que emite un radio de ruido proporcional a su velocidad (radio de andar 2 m, radio de correr 3 m a partir de 3.5 m/s). Este componente es el puente entre el sistema de movimiento y la percepción auditiva de los guardias mediante **HearingSensor**.

Comunicación entre agentes

El sistema implementa el protocolo **FIPA Contract Net**. Una conversación atraviesa tres fases:

Fase 0 — Reclutamiento El Iniciador envía un **REQUEST** con la tarea a todos los agentes registrados. Cada destinatario responde **AGREE** o **REFUSE** antes del **replyBy**. El guardián siempre responde **REFUSE** a cualquier **REQUEST** porque no puede moverse al interior. El Iniciador construye la lista de voluntarios.

Fase 1 — Negociación El Iniciador envía un **CFP** con la posición de referencia a todos los voluntarios. Cada voluntario calcula su coste como distancia euclídea a esa posición y envía un **PROPOSE**. El Iniciador selecciona la propuesta de menor coste y envía **ACCEPT_PROPOSAL** al ganador y **REJECT_PROPOSAL** a los perdedores.

Fase 2 — Ejecución y cierre El ganador entra en estado **PART_EXECUTING**, ejecuta el behaviour comprometido y al terminar envía **INFORM_RESULT** sólo al Iniciador. Para difundir el resultado al resto, el ganador hace un **INFORM** (fuera del Contract Net) a todos los agentes interesados. Cada receptor reinicia su timer al recibir cualquiera de los dos mensajes.

Excepciones y errores

- **NOT_UNDERSTOOD**: lo gestiona automáticamente la capa social y la deliberativa nunca lo ve.
- **FAILURE**: se trata como un **INFORM_RESULT** con **exito=false**.
- **CANCEL**: un Participante en **PART_EXECUTING** aborta y vuelve a patrullar.
- Timeouts de fase (**timeoutFase**): cierran la conversación si no se reciben todas las respuestas a tiempo.
- **INFORM** de búsqueda coordinada: cuando un patrullero pierde al ladrón, emite directamente un **INFORM** a todos los demás patrulleros con la última posición conocida, y él mismo calcula su propio sector sin esperar respuesta. No hay negociación ni ganador único: cada receptor calcula autónomamente su sector de búsqueda y se dirige a él. Este uso del **INFORM** fuera del flujo Contract Net es deliberado: la coordinación emerge de las posiciones relativas sin necesidad de propuestas.

Los mensajes se crean, envían y reciben a través de **AgenteFIPAACL**: la factoría **CrearMensaje** construye el **ACLMessage** con **sender** ya relleno y un GUID como **conversationId**. El envío es siempre **unicast** — el mensaje se deposita directamente en la cola **inbox** del destinatario. El multicast se realiza mediante bucles explícitos en la capa social. Cada agente drena su **inbox** en **Update** con un límite de **mensajesPorFrame** mensajes por frame, evitando congelar el juego ante avalanchas de mensajes.

Formato del contenido de los lenguajes

Estructura de **ACLMessage**

Cada mensaje es una instancia de **ACLMessage**, clase plana con los siguientes campos:

- **performative (string)**: acto comunicativo. Siempre se rellena.
- **sender (string)**: identificador del agente emisor, relleno automáticamente por **CrearMensaje**.
- **receivers (List<string>)**: identificador del destinatario (unicast; en la práctica siempre contiene un elemento).

- **content (object)**: payload del mensaje. Su tipo real lo conoce sólo la capa social según el **performative**.
- **conversationId (string)**: GUID asignado por el Iniciador al abrir la conversación. Todos los mensajes de una misma negociación comparten este identificador.
- **inReplyTo (string)**: **conversationId** del mensaje al que responde, para encadenar turnos.
- **replyBy (float)**: tiempo absoluto de juego (**Time.time**) antes del cual se espera respuesta; usado para el deadline de CFP.

Se han omitido deliberadamente los campos **language**, **ontology** y **protocol** del estándar FIPA: el primero es redundante porque el **content** siempre es un struct C# tipado conocido por el receptor según el **performative**; el segundo porque sólo existe un dominio semántico (el juego); el tercero también al usar solamente contractnet.

Performatives

SkeletonSocialLayer define las siguientes constantes **string** que representan todos los actos comunicativos del protocolo:

REQUEST, AGREE, REFUSE, CFP, PROPOSE,
ACCEPT_PROPOSAL, REJECT_PROPOSAL, INFORM_RESULT,
INFORM, FAILURE, NOT_UNDERSTOOD, CANCEL

CameraSocialLayer define únicamente los performatives necesarios para el rol de Iniciador (REQUEST, CFP, ACCEPT_PROPOSAL, REJECT_PROPOSAL, REFUSE), con los mismos valores de cadena para garantizar la interoperabilidad con los guardias.

Estructuras de contenido

El campo **content** de los mensajes que llevan datos propios es un struct tipado. Los performatives sin datos propios (AGREE, REFUSE, ACCEPT_PROPOSAL, REJECT_PROPOSAL, FAILURE, CANCEL, NOT_UNDERSTOOD) llevan **content = null**; el performative junto al **conversationId** es suficiente para identificar su significado.

- **ContentRequest**: campo **string** tarea. Usado en REQUEST.

- **ContentCFP**: campos `string tarea` y `Vector3 posicionReferencia`. Usado en CFP.
- **ContentPropose**: campo `float coste`. Usado en PROPOSE.
- **ContentInform**: campos `string tarea`, `bool exito` y `object datos`. Usado en `INFORM_RESULT` e `INFORM`.
- **ContentBusqueda**: campo `Vector3 ultimaPosicion`. Usado en el `INFORM` de búsqueda coordinada.

Procesamiento del contenido

Al recibir un mensaje, `ProcesarMensaje(ACLMessage)` en cada capa social aplica *pattern matching* sobre el par (`performative`, `tipo de content`). Si el `performative` es desconocido o el `content` no es del tipo esperado, la capa social genera automáticamente un `NOT_UNDERSTOOD` al emisor sin notificar a la deliberativa. Los `NOT_UNDERSTOOD` entrantes se ignoran silenciosamente para evitar bucles. Todo el vocabulario y la lógica de decodificación residen exclusivamente en la capa social; la deliberativa sólo recibe eventos en términos del dominio del juego (`OnRequestReceived`, `OnProposalReceived`, etc.) sin ver nunca un `ACLMessage`.

Estrategia Multiagente

La estrategia cooperativa del sistema tiene un objetivo común: impedir que el ladrón escape con el botín. Para lograrlo, los agentes coordinan sus acciones mediante el protocolo Contract Net, con roles diferenciados y comportamientos emergentes que se adaptan al estado de la partida.

Roles diferenciados

- **Cámara — Iniciador exclusivo**: detecta al ladrón y delega en los patrulleros la investigación. Nunca participa en contratos ajenos.
- **Guardián — Iniciador exclusivo**: detecta al ladrón en el exterior y coordina a los patrulleros para cubrir el punto de entrada. Siempre rechaza participar en contratos porque no puede moverse al interior.
- **Patrullero — Iniciador y Participante**: es el agente polivalente. Puede lanzar protocolos por timer (comprobación del cofre) o por percepción directa (bloqueo de salida, búsqueda coordinada), y también

ejecutar tareas asignadas por otros agentes (investigar avistamiento, acudir al punto de entrada, bloquear la salida).

Mecanismos de coordinación

La coordinación se implementa mediante dos mecanismos complementarios:

1. **Contract Net**: selección del agente más adecuado (menor coste = menor distancia) para ejecutar una tarea. Se usa cuando hay un ganador único: `comprobarCofre`, `investigarAvistamiento`, `ladronAvisado`, `bloquearSalida`.
2. **INFORM broadcast**: difusión de información sin negociación. Se usa cuando todos los agentes deben actuar: búsqueda coordinada tras perder al ladrón (cada uno va a su sector), activación del modo alerta al confirmar el robo del cofre.

Comportamientos coordinados emergentes

Comprobación periódica del cofre El timer coordinado de la capa social asegura que cada cierto tiempo aleatorio uno de los agentes lance un Contract Net con tarea `comprobarCofre`. El más cercano (menor coste) va al cofre. Si lo encuentra robado, hace un INFORM broadcast y todos los patrulleros activan permanentemente el **modo alerta** (subida de velocidad de patrulla mediante `ActivarModoAlerta()`).

Investigación de avistamientos de cámara Cuando una cámara ve al ladrón, lanza `investigarAvistamiento`. El patrullero más cercano se desplaza con `GoToPositionBehaviour`. Al llegar, el `MovementSensor.OnDestinoAlcanzado` dispara automáticamente `EscanearEntorno()` y `PatrolBehaviour` recalcula el destino desde la nueva posición, dejando al guardia patrullando indefinidamente la zona investigada sin necesidad de código adicional.

Alerta desde guardia estático Cuando un guardián exterior ve al ladrón, lanza `ladronAvisado` con la posición de su `puntoEntrada` (un Transform dentro del NavMesh). El patrullero más cercano se desplaza allí y queda patrullando esa zona, exactamente como en `investigarAvistamiento`.

Bloqueo de la zona de salida El protocolo `bloquearSalida` puede activarse por dos vías complementarias: por visión directa del cofre vacío (`OnCofresDesaparecido`) o al recibir el INFORM broadcast del ganador de

`comprobarCofre` que confirma el robo. En ambos casos, el primer patrullero libre lanza el Contract Net con la posición de la `ZonaSalida`; el más cercano se desplaza allí y queda patrullando la salida. Este efecto ocurre en paralelo al modo alerta, que también se activa al recibir el mismo `INFORM` broadcast.

Búsqueda coordinada tras perder al ladrón Cuando un patrullero pierde al ladrón, emite un `INFORM` con la última posición conocida. Todos los patrulleros (incluido el emisor) calculan su sector de forma autónoma: consultan `AgenteFIPAACL.Todos`, obtienen su índice entre los patrulleros y se les asigna el ángulo $\text{índice} \times (360/\text{total})$. El destino se corrige con `NavMesh.SamplePosition` para garantizar que cae dentro del `NavMesh`. Aquí se usa deliberadamente un `INFORM` en lugar de un Contract Net porque no hay un ganador único: todos participan con un sector distinto y la dispersión emerge de las posiciones relativas sin propuestas.

Decisiones globales adaptativas

El sistema adapta su comportamiento global en función del estado de la partida de forma emergente, sin un controlador centralizado:

- **Antes del robo:** los agentes se coordinan periódicamente para verificar el estado del cofre y responden a avistamientos del ladrón de forma reactiva (guardián, cámara) o proactiva (timer).
- **Tras el robo confirmado:** todos los patrulleros aumentan la velocidad de patrulla (modo alerta) y el más cercano a la salida se desplaza a bloquearla, incrementando la presión sobre el ladrón.
- **Tras perder al ladrón:** en lugar de converger al mismo punto, los patrulleros se dispersan en sectores para cubrir la mayor área posible, maximizando la probabilidad de redetección.